

Ingeniería en Computación — 4to Año UNRaf

COMUNICACIÓN DE DATOS

TAREA N° 1 — COMUNICACIONES CABLEADAS

*Implementación de comunicación paralela de 8 bits y
comunicación serie bidireccional UART entre microcontroladores Arduino UNO*

Autores: Bevilacqua Francisco – Clement Sebastián

Materia: Comunicación de Datos

Fecha: 26 de Abril 2026

Versión: Bus Paralelo v4.0.0 | Serie v1.0.0

Resumen

El presente informe documenta el diseño, desarrollo e implementación de dos sistemas de comunicación cableada entre pares de microcontroladores Arduino UNO (ATmega328P). El primer ejercicio establece una comunicación paralela unidireccional de 8 bits mediante un bus de datos con señal de strobe (**STR**), incluyendo un protocolo de tramas de cinco bytes con detección de errores XOR. El segundo ejercicio implementa una comunicación serie bidireccional UART mediante la biblioteca **SoftwareSerial**, con un protocolo de handshake, comandos tipificados y detección de errores por suma módulo 256.

Ambos sistemas incorporan sistemas de logging detallados a través del monitor serie del IDE de Arduino, que permitieron diagnosticar y resolver errores durante el desarrollo. En particular, el ejercicio 1 atravesó cuatro iteraciones de firmware hasta alcanzar la versión v4.0.0, la cual resolvió un problema crítico de interferencia capacitiva/inductiva (crosstalk) entre las señales PWM del LED RGB y las líneas del bus de datos, reasignando los pines del bus a entradas/salidas digitales puras sin función de timer.

Abstract

This report documents the design, development and implementation of two wired communication systems between pairs of Arduino UNO (ATmega328P) microcontrollers. The first exercise establishes an 8-bit unidirectional parallel communication using a data bus with a strobe signal (STR), including a five-byte frame protocol with XOR error detection. The second exercise implements a bidirectional UART serial communication via the SoftwareSerial library, with a handshake protocol, typed commands and modulo-256 sum error detection. Both systems include detailed logging through the Arduino IDE serial monitor, which enabled diagnosis and correction of errors during development.

Palabras clave: comunicación paralela, bus de datos, strobe, checksum XOR, UART, SoftwareSerial, handshake, ACK/NACK, PWM, crosstalk, Arduino UNO, ATmega328P.

Índice

1. Introducción	4
2. Ejercicio 1: Comunicación Paralela de 8 Bits	5
2.1 Descripción del sistema y requisitos	5
2.2 Protocolo de comunicación paralela	6
2.3 Timing del protocolo y cronograma	7
2.4 Esquema de bloques funcionales	9
2.5 Implementación del firmware — Dispositivo 1 (Emisor)	10
2.6 Implementación del firmware — Dispositivo 2 (Receptor)	13
2.7 Checksum XOR: fundamento teórico y justificación	15
2.8 Sistema de logging y diagnóstico	16
2.9 Evolución del firmware: de v1 a v4 — Análisis forense de errores	17
2.10 Conexión eléctrica y esquema de circuito	20
2.11 Resultados	21
3. Ejercicio 2: Comunicación Serie Bidireccional UART	22
3.1 Descripción del sistema y requisitos	22
3.2 El periférico UART y la biblioteca SoftwareSerial	23
3.3 Protocolo serie: tramas y handshake	24
3.4 Máquina de estados de D1 y D2	26
3.5 Checksum por suma módulo 256: fundamento teórico	27
3.6 Implementación del firmware — Dispositivo 1 (Maestro)	28
3.7 Implementación del firmware — Dispositivo 2 (Receptor/Esclavo)	30
3.8 Sistema de logging y diagnóstico	32
3.9 Comparación con protocolo alternativo	33
3.10 Conexión eléctrica y esquema de circuito	34
3.11 Resultados	35

4. Comparación entre comunicación paralela y serie	36
5. Dificultades encontradas y soluciones aplicadas	37
6. Conclusiones	38
7. Referencias bibliográficas	39
8. Link a videos mostrando funcionamiento.....	

1. Introducción

La comunicación de datos constituye el eje central de cualquier sistema de información distribuido. Según Stallings (2022), un sistema de comunicación de datos puede analizarse a través de cinco bloques funcionales fundamentales: fuente, transmisor, medio de transmisión, receptor y destino. La correcta comprensión de cada bloque y su interacción es requisito indispensable para el diseño de protocolos robustos.

Tanenbaum y Wetherall (2011) distinguen dos grandes paradigmas de transmisión: la comunicación **paralela**, en la que múltiples bits se transmiten simultáneamente sobre múltiples conductores, y la comunicación **serie**, en la que los bits se envían secuencialmente sobre un único conductor. Cada paradigma presenta ventajas y desventajas en términos de velocidad, consumo de pines, inmunidad al ruido y distancia de operación efectiva.

El presente trabajo implementa ambos paradigmas sobre la plataforma Arduino UNO (microcontrolador ATmega328P de Atmel/Microchip) con el objetivo de contrastar sus características y validar experimentalmente los conceptos teóricos. Se documenta el proceso completo de desarrollo, incluyendo los errores encontrados en versiones preliminares y las soluciones técnicas adoptadas, siguiendo la metodología del análisis forense de logs que permitió identificar la causa raíz de los fallos.

1.1 Plataforma de desarrollo: Arduino UNO — ATmega328P

El Arduino UNO está basado en el microcontrolador ATmega328P, un dispositivo de 8 bits con arquitectura Harvard modificada que opera a 16 MHz. Sus características relevantes para este trabajo son:

- **14 pines de E/S digitales**, de los cuales 6 son aptos para PWM (Pulse Width Modulation) mediante timers de hardware: pines 3 y 11 (Timer2), pines 9 y 10 (Timer1), y pines 5 y 6 (Timer0).
- **6 entradas analógicas** (A0–A5) que también pueden usarse como GPIO digital puro. En el ATmega328P, las entradas A0–A5 no tienen función de timer asociada, lo que resulta crucial para el ejercicio 1.
- **1 puerto UART hardware** (pines 0/RX y 1/TX), compartido con el convertidor USB-Serie del IDE de Arduino.
- **SoftwareSerial**: biblioteca de Arduino que emula un puerto UART por software en cualquier par de pines GPIO mediante interrupciones.

- **Timers de hardware:** Timer0 (8 bits, pines 5 y 6), Timer1 (16 bits, pines 9 y 10), Timer2 (8 bits, pines 3 y 11). La función `analogWrite()` utiliza estos timers para generar señales PWM.

2. Ejercicio 1: Comunicación Paralela de 8 Bits

2.1 Descripción del sistema y requisitos

El ejercicio 1 establece una comunicación paralela unidireccional de 8 bits entre dos Arduino UNO. El Dispositivo 1 (D1) actúa como **emisor** y dispone de dos pulsadores: BTN_ONOFF (enciende/apaga el LED RGB) y BTN_CAMBIAR (genera un color RGB aleatorio). El Dispositivo 2 (D2) actúa como **receptor** y controla un LED RGB tipo cátodo común conectado a sus salidas PWM.

Cuando el usuario presiona un botón en D1, este construye una trama de 5 bytes y la transmite por el bus de datos paralelo. D2 monitorea continuamente la señal de strobe (STR) mediante polling en su loop principal; al detectar un flanco ascendente, lee el byte presente en el bus y lo almacena en un buffer de recepción. Una vez recibidas las 5 tramas, verifica la integridad mediante el checksum y, si es correcta, ejecuta el comando sobre el LED RGB.

2.2 Protocolo de comunicación paralela

El protocolo define una transmisión compuesta por 5 tramas consecutivas, cada una transportando 1 byte (8 bits) sobre el bus paralelo. La señal de strobe (STR) actúa como señal de validez: STR=LOW indica bus inactivo o en transición; STR=HIGH indica que el dato en el bus es estable y válido para ser leído.

Trama	Campo	Descripción
T0	Control	Bit 7 (CMD): 0 = ON/OFF 1 = COLOR. Bit 0 (STATE): 1 = ENCENDER 0 = APAGAR (solo si CMD=0) Bits 6–1: reservados, siempre 0
T1	Rojo (R)	Intensidad canal rojo (0–255). 0x00 si CMD=ON/OFF
T2	Verde (G)	Intensidad canal verde (0–255). 0x00 si CMD=ON/OFF
T3	Azul (B)	Intensidad canal azul (0–255). 0x00 si CMD=ON/OFF
T4	Checksum	XOR de T0, T1, T2 y T3: $CHK = T0 \oplus T1 \oplus T2 \oplus T3$

La selección de 5 bytes (y no más) obedece al mínimo necesario para transportar el tipo de comando (1 byte), los tres canales del color (3 bytes) y el checksum de integridad (1 byte). Esta estructura es análoga a la de protocolos industriales de bus paralelo como el IEEE-488 (GPIB), donde cada dato va acompañado de señales de control (Stallings, 2022).

2.3 Timing del protocolo y cronograma

El timing de cada trama es el aspecto más crítico del protocolo paralelo, ya que debe garantizar que D2 lea el bus única y exclusivamente cuando los 8 bits son estables. La versión 4.0.0 del firmware define cuatro intervalos de tiempo:

Parámetro	Valor	Justificación
SETUP_TIME_MS	15 ms	Tiempo desde que D1 escribe el bus hasta que eleva STR. Permite que los 8 pines se estabilicen eléctricamente superando la capacitancia parásita de los cables jumpers que conectan D1 y D2.
STROBE_HIGH_MS	8 ms	Ventana de lectura: STR permanece en HIGH durante 8 ms. D2 ejecuta su loop principal a velocidad total (sin Serial dentro del loop), por lo que detecta el flanco en <1 ms. Los 8 ms extra absorben variaciones de carga del loop (log post-trama).
HOLD_TIME_MS	8 ms	Tiempo desde que STR baja hasta que D1 limpia el bus. D2 puede tardar hasta ~3 ms en registrar el dato internamente después del flanco descendente. 8 ms brinda margen 2.5x.
INTER_FRAME_MS	5 ms	Pausa entre tramas consecutivas. Da tiempo a D2 para ejecutar analogWrite() del LED (acción de la trama anterior) antes de que el bus cambie al siguiente dato.

La duración total por trama es: $15 + 8 + 8 + 5 = \mathbf{36\ ms}$. La transmisión completa de 5 tramas dura aproximadamente **180 ms**. El timeout configurado en D2 para esperar cada trama es de **1000 ms**, es decir 27,7 veces el tiempo esperado, lo que otorga un margen extremadamente generoso para absorber cualquier variación de carga del sistema.

Cronograma temporal de una trama (v4.0.0)

Fase 1 — SETUP (15 ms): D1 escribe los 8 bits en el bus. STR=LOW. D2 ignora el bus.

Fase 2 — STROBE HIGH (8 ms): D1 eleva STR. D2 detecta el flanco y ejecuta leerBus().

Fase 3 — HOLD (8 ms): D1 baja STR. D2 termina de registrar el dato.

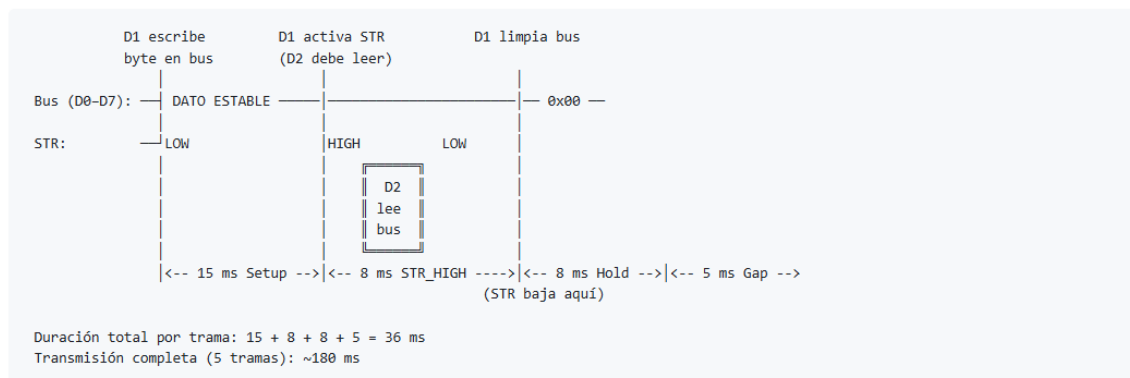
Fase 4 — INTER-FRAME (5 ms): D1 pone el bus en 0x00. D2 procesa el dato (LED, log).

0 ms 15 ms 23 ms 31 ms 36 ms
 |-----|-----|-----|-----|
 SETUP STR=HI STR=LO LIMPIA INICIO T+1
 (bus OK) (D2 lee) (hold) (gap)

Este diseño sigue el principio de **setup-and-hold** ampliamente documentado en la literatura de diseño digital (Tanenbaum & Wetherall, 2011). El setup time garantiza que la señal de datos se haya estabilizado antes del flanco activo del reloj (o en este caso, de la señal de strobe), mientras que el hold time garantiza que el dato se mantiene válido durante el tiempo suficiente para que el flip-flop del receptor capture el valor correctamente.

El criterio de diseño para evitar lecturas erróneas durante las transiciones del bus es la **separación temporal estricta entre fases**: el bus nunca está cambiando al mismo tiempo que STR está en HIGH. D1 garantiza esto mediante la secuencia: escribir bus → esperar SETUP_TIME_MS → elevar STR → esperar STROBE_HIGH_MS → bajar STR → esperar HOLD_TIME_MS → limpiar bus. No hay ninguna condición de carrera posible siempre que D2 solo lea el bus cuando STR=HIGH, que es exactamente lo que implementa la función `esperarStrobe()`.

3.3 Timing por Trama (Cronograma)



Criterio de diseño para evitar lecturas erróneas durante transiciones:

D2 solo lee el bus cuando detecta el **flanco ascendente** de STR (LOW→HIGH). En ese momento, D1 ya esperó 15 ms desde que escribió el bus (setup time), garantizando que todos los pines estabilizaron su nivel lógico antes de habilitar la lectura. La señal STR actúa como señal de habilitación (*enable*): sin ella, D2 ignora completamente el estado del bus. Esto elimina lecturas en transición.

2.4 Esquema de bloques funcionales

Siguiendo el modelo de bloques funcionales de Stallings (2022), el sistema de comunicación paralela puede describirse de la siguiente manera:

- **Fuente de información:** los pulsadores BTN_ONOFF y BTN_CAMBIAR del Dispositivo 1 generan los eventos que originan la información a transmitir.
- **Transmisor:** el firmware del Dispositivo 1 (funciones `enviarTramaOnOff()` y `enviarTramaColor()`) codifica la información en la estructura de trama de 5 bytes, la escribe en el bus paralelo y genera la señal de strobe.
- **Canal de transmisión:** el bus físico compuesto por 9 conductores (8 de datos + 1 de strobe) más el cable de referencia de masa (GND común). En el protoboard,

este canal es susceptible a interferencias capacitivas/inductivas entre cables adyacentes.

- **Receptor:** el firmware del Dispositivo 2 (funciones `leerBus()` y `recibirTransmision()`) detecta el flanco ascendente de STR, lee los 8 bits del bus y almacena las 5 tramas en el buffer de recepción.
- **Destino de la información:** la función `procesarTrama()` verifica el checksum, interpreta el comando y ejecuta `aplicarLED()` o `apagarLED()`, produciendo el resultado final: el cambio de estado del LED RGB.

2. Esquema de Bloques del Sistema de Comunicación

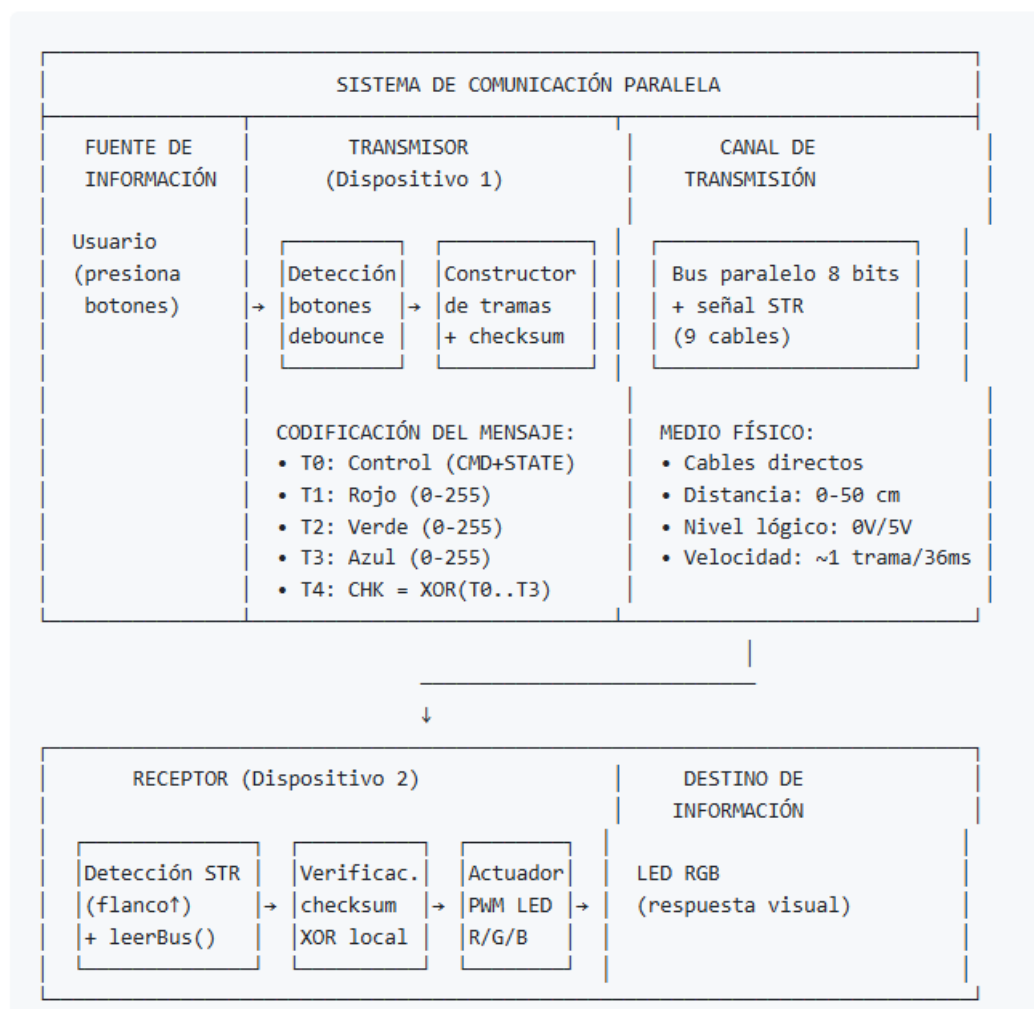


FOTO: Esquema de bloques funcionales del sistema de comunicación paralela.

2.5 Implementación del firmware — Dispositivo 1 (Emisor)

El firmware del Dispositivo 1 (archivo `dispositivo1_paralelo_v4.ino`) implementa las siguientes funciones principales:

2.5.1 Configuración del bus de datos

La función `configurarBus()` inicializa los 8 pines del bus como salidas (`OUTPUT`) con nivel lógico LOW, y el pin de strobe también como salida en LOW. La asignación de pines v4.0.0 es:

```
// Asignación de pines del bus v4.0.0
PIN_D0 = 2    // Digital puro
PIN_D1 = A2    // v4: antes pin 3 (OC2B/Timer2) → A2 (GPIO puro)
PIN_D2 = A3    // v4: antes pin 4 (adyacente OC2B) → A3 (GPIO puro)
PIN_D3 = 5    // Digital puro
PIN_D4 = 6    // Digital puro
PIN_D5 = 7    // Digital puro
PIN_D6 = 8    // Digital puro
PIN_D7 = A0    // Analógico como digital puro
PIN_STR = A1   // Strobe: señal de validez del bus
```

2.5.2 Escritura en el bus con verificación (nuevo en v4)

La función `escribirByte(dato)` escribe los 8 bits del dato en los pines del bus mediante `digitalWrite()`. Una innovación introducida en v4 es la **dobles lectura de verificación**: después de escribir, el firmware espera 50 μ s y lee de vuelta el estado de cada pin. Si la lectura no coincide con el valor escrito, reintenta hasta `WRITE_RETRIES` (3) veces. Este mecanismo detecta y corrige glitches de escritura causados por capacitancia parásita en cables largos de protoboard.

2.5.3 Activación del strobe y timing crítico

La función `activarStrobe()` implementa la secuencia de timing crítica. Deliberadamente, no contiene ningún llamado a `Serial.print()`. Esta decisión de diseño es fundamental: la transmisión UART del Serial Hardware (115200 bps) puede introducir jitter de hasta varios milisegundos en los GPIO durante la escritura, lo que podría perturbar la señal STR o los datos del bus.

```
void activarStrobe() {
    delay(SETUP_TIME_MS);    // 15 ms: bus estable antes del flanco
    digitalWrite(PIN_STR, HIGH); // Flanco ascendente: D2 lee el bus
    delay(STROBE_HIGH_MS);   // 8 ms: ventana de lectura
    digitalWrite(PIN_STR, LOW); // Flanco descendente: fin de ventana
    delay(HOLD_TIME_MS);     // 8 ms: margen post-lectura
}
```

2.5.4 Transmisión en dos fases: zona crítica y logging

La función `transmitirTrama()` implementa un principio de diseño clave: la separación estricta entre la **fase de transmisión hardware** (sin ningún `Serial.print()`) y la **fase de logging** (posterior a que el bus quedó libre). En la fase 1, se transmiten todos los bytes;

en la fase 2, se imprimen los datos enviados con detalle. De este modo, el log no introduce latencia en el protocolo de tiempo real.

2.5.5 Detección de pulsadores con debounce por software

La función `leerBoton()` implementa el anti-rebote (debounce) por software mediante timestamps: solo reconoce un nuevo evento de pulsación si han transcurrido al menos `DEBOUNCE_MS` (50 ms) desde el último evento válido. Los pulsadores se configuran con `INPUT_PULLUP`, de modo que el nivel lógico es HIGH en reposo y LOW cuando se presiona, sin necesidad de resistencias externas.

2.6 Implementación del firmware — Dispositivo 2 (Receptor)

El firmware del Dispositivo 2 (archivo `dispositivo2_paralelo_v4.ino`) implementa las siguientes funciones principales:

2.6.1 Polling de la señal STR y detección de flanco

El loop principal del Dispositivo 2 ejecuta `digitalRead(PIN_STR)` continuamente, comparando el valor actual con el estado anterior (variable `strobeAnterior`). Al detectar la transición LOW→HIGH, llama inmediatamente a `leerBus()` para capturar T0. Esta técnica de polling garantiza la máxima velocidad de detección: sin llamadas a Serial en el loop, el ATmega328P puede ejecutar varios miles de iteraciones por segundo, lo que hace prácticamente imposible perder un flanco de 8 ms de duración.

2.6.2 Lectura del bus con doble verificación (nuevo en v4)

Análogamente al emisor, la función `leerBus()` del receptor implementa doble lectura de verificación: realiza dos lecturas consecutivas del bus con 50 μ s de espera entre ellas. Si ambas lecturas coinciden, el dato se considera estable y válido. Si difieren, se reintenta hasta `READ_RETRIES` (3) veces. Este mecanismo es especialmente efectivo contra glitches de corta duración (<50 μ s) causados por crosstalk o transiciones de bus.

2.6.3 Recepción de las tramas T1–T4

La función `recibirTransmision()` espera el flanco ascendente de STR para cada una de las tramas T1–T4, utilizando `esperarStrobe()` con un timeout de 1000 ms. Al detectar cada flanco, llama a `leerBus()` y almacena el resultado en `bufferRx[]` junto con el timestamp (`millis()`). Los timestamps permiten calcular los intervalos inter-trama en el log, que sirven como diagnóstico del timing de D1.

2.6.4 Procesamiento y control del LED RGB

Una vez recibidas las 5 tramas, `procesarTrama()` verifica el checksum XOR. Si es correcto, interpreta el campo CMD del byte T0: si CMD=0 (ON/OFF), lee el bit STATE y

llama a `aplicarLED()` o `apagarLED()`; si `CMD=1` (COLOR), extrae `R=T1`, `G=T2`, `B=T3` y llama a `aplicarLED(R,G,B)`. La función `aplicarLED()` llama a `analogWrite()` sobre los pines 9, 10 y 11 para generar las señales PWM que controlan la intensidad de cada canal del LED RGB.

2.7 Checksum XOR: fundamento teórico y justificación

El checksum XOR implementado en el ejercicio 1 se calcula como: **$CHK = T0 \oplus T1 \oplus T2 \oplus T3$** . El operador XOR ($\oplus$) tiene la propiedad de que el resultado es 0 cuando todos los operandos son iguales, y es sensible a cualquier cambio de bit en cualquiera de los operandos. Esta propiedad lo hace útil como mecanismo de detección de errores de paridad.

La principal **ventaja del XOR** es su simplicidad computacional: es una operación de 1 ciclo de reloj en cualquier CPU, sin riesgo de desbordamiento. Esto es especialmente relevante en microcontroladores de 8 bits como el ATmega328P, donde las operaciones aritméticas de mayor precisión consumen más ciclos.

La principal **limitación del XOR** es que no detecta errores que se cancelan entre sí: por ejemplo, si dos bits idénticos en posiciones idénticas de dos bytes distintos se alteran simultáneamente, el XOR no detecta el error. Tampoco detecta reordenamiento de bytes (si `T1` y `T2` se intercambian, el XOR es idéntico). Para mayor robustez se requeriría CRC-16 o CRC-32 (Stallings, 2022), pero para los efectos de este protocolo educativo el XOR es más que suficiente.

En el ejercicio 1, el XOR se justifica además porque la interferencia observada fue siempre en bits individuales (`D1` y `D2` del bus, ver sección 2.9), que son exactamente el tipo de error que el XOR detecta con 100% de eficacia.

2.8 Sistema de logging y diagnóstico

Ambos firmwares implementan un sistema de logging detallado a través del puerto UART hardware (pines 0/1) a 115200 bps, accesible desde el Serial Monitor del IDE de Arduino. Este logging fue instrumental para el diagnóstico de errores en las versiones anteriores.

Componentes del sistema de logging:

- **Log de cabecera TX:** al inicio de cada transmisión, imprime el número de TX, el tipo de trama y una tabla con el valor hex, decimal y binario de cada byte, más el cálculo paso a paso del XOR.

- **Log post-transmisión:** después de que el bus quedó libre, confirma cada byte enviado con su representación binaria y los parámetros de timing utilizados.
- **Log de recepción:** tabla completa de las 5 tramas recibidas con valores hex/dec/bin y el tiempo relativo desde T0 en ms (para diagnóstico de timing de D1).
- **Log de checksum:** muestra el cálculo XOR bit a bit, la comparación entre CHK calculado y CHK recibido, e identifica exactamente qué bits difieren (si los hay).
- **Log de error:** en caso de checksum incorrecto, imprime la diferencia binaria, identifica los bits en error (ej: "D1 D2" = bits 1 y 2) y proporciona diagnóstico específico para v4.
- **Contadores de sesión:** TX total, RX total, errores de checksum, errores de timeout, reintentos de escritura y lectura, y tasa de éxito porcentual.

La **decisión de diseño más importante** del sistema de logging es que **ningún `Serial.print()` se ejecuta durante la fase de transmisión hardware**. Esto se implementa en `transmitirTrama()` mediante la división en dos fases explícitas: la fase crítica (sin `Serial`) y la fase de logging (post-transmisión). Análogamente, en D2, `recibirTransmision()` no contiene ningún llamado a `Serial`; todo el logging se centraliza en `procesarTrama()`.

Apartado de resultados — Logs del ejercicio 1:

Versión 4.

Log completo de una transmisión COLOR en Dispositivo 1 (Serial Monitor TX)

```
-----
[BTN] CAMBIAR detectado | Color RGB: R=228 G=237 B=59
-----

TX #2 | Tipo: COLOR RGB
Trama | Hex | Dec | Binario | Descripción
-----|-----|-----|-----|-----
T0 | 0x80 | 128 | 1000 0000 | CMD=COLOR (bit7=1)
T1 | 0xE4 | 228 | 1110 0100 | Canal Rojo R = 228
T2 | 0xED | 237 | 1110 1101 | Canal Verde G = 237
T3 | 0x3B | 59 | 0011 1011 | Canal Azul B = 59
[CHK] XOR: 0x80 ^ 0xE4 ^ 0xED ^ 0x3B = 0xB2
      Bit a bit (MSB a LSB):
      T0 = 1000 0000
      T1 = 1110 0100
      T2 = 1110 1101
      T3 = 0011 1011
      CHK= 1011 0010
T4 | 0xB2 | 178 | 1011 0010 | Checksum XOR(T0^T1^T2^T3)
--- Iniciando transmisión (logging suspendido) ---
--- Tramas enviadas (log post-transmission) ---
Timing: Setup=15ms | STR=8ms | Hold=8ms | Gap=5ms
[T0] Valor enviado: 0x80 | 1000 0000
[T1] Valor enviado: 0xE4 | 1110 0100
[T2] Valor enviado: 0xED | 1110 1101
[T3] Valor enviado: 0x3B | 0011 1011
[T4] Valor enviado: 0xB2 | 1011 0010 | (última trama - CHK)
--- Transmisión completada ---
Bus = 0x00 | STR = LOW | Canal libre
-----
```

Log completo de una recepción correcta en Dispositivo 2 (Serial Monitor RX) — Checksum OK

```
-----
RX #2 | Tabla de tramas recibidas:
Trama | Hex | Dec | Binario | t_rel(ms)
-----|-----|-----|-----|-----
T0-Control | 0x80 | 128 | 1000 0000 | +0 ms
T1-Rojo | 0xE4 | 228 | 1110 0100 | +36 ms
T2-Verde | 0xED | 237 | 1110 1101 | +72 ms
T3-Azul | 0x3B | 59 | 0011 1011 | +109 ms
T4-Checksum | 0xB2 | 178 | 1011 0010 | +145 ms
Intervalos entre tramas (diagnostico timing D1):
T0 → T1 : 36 ms
T1 → T2 : 36 ms
T2 → T3 : 37 ms
T3 → T4 : 36 ms
CHK calculado local : 0x80 ^ 0xE4 ^ 0xED ^ 0x3B = 0xB2
      Bit a bit (MSB→LSB):
      T0 = 1000 0000
      T1 = 1110 0100
      T2 = 1110 1101
      T3 = 0011 1011
      CHK(calc) = 1011 0010 (0xB2)
      CHK(recv) = 1011 0010 (0xB2)
      Diferencia = .... (sin diferencia - OK)
CHK recibido (T4) : 0xB2
CHK calculado (local): 0xB2

[CHK] OK → Ejecutando comando...

[CMD] COLOR → R=228 G=237 B=59
[LED] PWM aplicado: R=228 G=237 B=59
-----
```

Versión 3:

Log de envío en Dispositivo 1 durante Version 3 (antes de la corrección)

```
Ejemplo- ( DETALLE CURIOSO: FALLA EN TRAMA DE CHECKSUM):
16:47:16.051 ->          Bit a bit (MSB a LSB):
16:47:16.051 ->          T0 = 1000 0000
16:47:16.051 ->          T1 = 1101 1011
16:47:16.051 ->          T2 = 1111 0011
16:47:16.051 ->          T3 = 0101 0011
16:47:16.051 ->          CHK= 1111 1011
16:47:16.051 -> T4      | 0xFB | 251 | Checksum XOR(T0^T1^T2^T3)
16:47:16.051 -> --- Iniciando transmision ---
16:47:16.082 -> [STR T0] Setup=10ms -> STR_HIGH=5ms -> Hold=2ms -> Bus=0x00
16:47:16.082 -> [STR T1] Setup=10ms -> STR_HIGH=5ms -> Hold=2ms -> Bus=0x00
16:47:16.115 -> [STR T2] Setup=10ms -> STR_HIGH=5ms -> Hold=2ms -> Bus=0x00
16:47:16.147 -> [STR T3] Setup=10ms -> STR_HIGH=5ms -> Hold=2ms -> Bus=0x00
16:47:16.147 -> [STR T4] Setup=10ms -> STR_HIGH=5ms -> Hold=2ms -> Bus=0x00
16:47:16.147 -> --- Transmision completada ---
16:47:16.147 -> Bus = 0x00 | STR = LOW | Canal libre
16:47:16.180 -> -----
```

Log de error de checksum en Dispositivo 2 durante Version 3 (antes de la corrección) — bits D1 y D2 erróneos

```
Ejempl-
16:47:16.216 -> T0 = 1000 0000
16:47:16.216 -> T1 = 1101 1011
16:47:16.216 -> T2 = 1111 0011
16:47:16.217 -> T3 = 0101 0111
16:47:16.217 -> T4 = | 0xFF |
16:47:16.217 -> CHK = 1111 1011
16:47:16.217 -> CHK recibido (T4): 0xFF
16:47:16.217 -> CHK calculado (local): 0xFB
16:47:16.217 ->
16:47:16.217 -> [ERROR] CHECKSUM INCORRECTO -> Trama DESCARTADA.
16:47:16.217 -> CHK recibido (T4): 0xFF
16:47:16.217 -> CHK calculado (local): 0xFB
16:47:16.217 -> Diferencia (bits alterados): 0x04 bin=0000 0100
16:47:16.250 -> Bits en error: D2
16:47:16.250 -> Estado del LED NO modificado.
16:47:16.250 -> Errores de checksum acumulados: 10
16:47:16.250 -> DIAGNOSTICO:
16:47:16.250 -> 1. Verificar GND comun entre D1 y D2 (causa mas frecuente).
16:47:16.250 -> 2. Pin D7 (A0): es el bit de CMD, el mas critico.
16:47:16.250 -> Un D7 suelto o sin GND lee HIGH falso -> CHK incorrecto.
16:47:16.250 -> 3. Comparar tabla de arriba con TX #N del Serial de D1.
16:47:16.250 -> 4. Si solo falla el bit 7 (D7/A0): verificar ese cable.
16:47:16.289 -> -----
NOTA: Se observa que en este caso en particular, las tramas enviadas (control y RGB),
llegaron correctamente. Pero la trama recibida de checksum con un error. Por lo cual,
el protocolo implementado de control y manejo de errores fue eficiente, ya que si bien
la informacion respectiva al color RGB llego bien, se produjeron errores de indole eléctrico
en la trama de checksum.
```


2.9 Evolución del firmware: de v1 a v4 — Análisis de errores

El proceso de desarrollo del ejercicio 1 atravesó cuatro versiones de firmware hasta alcanzar una implementación estable. Cada versión representó una hipótesis diferente sobre la causa del error, y su refutación o confirmación fue guiada por el análisis de los logs del Serial Monitor.

2.9.1 Versión 1 — Prototipo inicial sin timing explícito

La primera versión implementaba la transmisión del bus directamente mediante `digitalWrite()` sin ningún delay de estabilización. El strobe se generaba inmediatamente después de escribir el bus. El resultado fue una alta tasa de errores de checksum, especialmente en las tramas donde algún bit differía de 0 (es decir, en tramas de COLOR pero no en tramas ON/OFF con $T1=T2=T3=0x00$). La causa identificada fue que D2 leía el bus en plena transición, capturando valores intermedios de los pines que aún no habían completado su carga RC hacia el nivel lógico correcto.

2.9.2 Versión 2 — Introducción de timing básico

La segunda versión introdujo un delay de 5 ms (`SETUP_TIME`) antes de elevar STR y un delay de 3 ms (`HOLD_TIME`) antes de limpiar el bus. La tasa de errores se redujo significativamente, pero persistía un error sistemático y reproducible: el checksum (T4) resultaba siempre incorrecto con una diferencia de exactamente 0x06 (bits D1 y D2 del bus), y solo en tramas COLOR. La regularidad de este patrón indicaba que no era un error aleatorio de ruido, sino un problema sistemático y determinista.

2.9.3 Versión 3 — Aumento de tiempos — Error persiste

La tercera versión aumentó los tiempos de setup y hold a 10 ms y 5 ms respectivamente, y agregó logging detallado. El análisis de los logs reveló el patrón definitivo: el error ocurría **siempre en T4 (CHK)**, **nunca en T0–T3**; la diferencia era **siempre 0x06 = 0b00000110** (exactamente bits D1 y D2); el timing inter-trama era regular (~31 ms); y el error ocurría **solo en tramas COLOR**, nunca en tramas ON/OFF cuyo payload era todo 0x00. Estos cuatro indicios apuntaban inequívocamente a una interferencia específica sobre los pines del bus D1 (pin 3) y D2 (pin 4).

Análisis del error de v3 — Cuatro indicios clave

1. ERROR SIEMPRE EN T4 (CHK): T0–T3 llegan con valores distintos a los enviados. El error aparece en bits D1 y D2.
2. DIFERENCIA EXACTA 0x06: Los bits que difieren son D1 (bit 1) y D2 (bit 2), es decir los pines 3 y 4.
3. TIMING REGULAR: El intervalo entre tramas es constante (~31 ms). No es causa de timing.

4. SOLO EN COLOR: Las tramas ON/OFF (payload=0x00) nunca fallan. El error depende del VALOR del byte.

Conclusión: El error es determinista y dependiente del estado de los pines D1/D2.
Causa: crosstalk PWM.

2.9.4 Causa raíz: crosstalk capacitivo/inductivo de las señales PWM

La causa raíz identificada fue la **interferencia electromagnética entre las señales PWM del LED RGB y las líneas del bus de datos**. En la asignación de pines de v3, el pin D1 del bus correspondía al pin 3 del Arduino, que es la salida OC2B del Timer2. El pin D2 del bus correspondía al pin 4, físicamente adyacente en el protoboard al pin 3 (OC2B).

Cuando el Dispositivo 2 ejecuta `analogWrite()` sobre los pines del LED RGB (9, 10 y 11), los timers Timer1 y Timer2 generan señales PWM a frecuencias de ~490 Hz (Timer2) y ~980 Hz (Timer1). Estas conmutaciones rápidas generan variaciones de corriente en los cables del protoboard que, por acoplamiento capacitivo e inductivo (crosstalk), inducen tensiones parásitas en los cables adyacentes. Los cables más afectados son los más cercanos a los cables PWM en el protoboard: precisamente D1 (pin 3 = OC2B) y D2 (pin 4 = adyacente a OC2B).

El mecanismo específico del error fue: mientras D1 realizaba el setup time antes de elevar STR para T4 (CHK), el Dispositivo 2 estaba ejecutando `analogWrite()` del LED RGB (acción de la trama T3 anterior). El PWM del Timer2 generaba glitches en los cables D1 y D2 del bus. Cuando D2 ejecutaba `leerBus()` durante el strobe, los pines 3 y 4 estaban perturbados, produciendo una lectura incorrecta con exactamente los bits 1 y 2 alterados.

La razón por la que el error ocurría **solo en T4 y solo en tramas COLOR** es consistente con esta causa: en tramas ON/OFF, T1=T2=T3=0x00, y el CHK es un valor simple que rara vez tiene los bits D1/D2 activos en estado diferente al que el glitch produce. En tramas COLOR, R, G y B son valores aleatorios de 0 a 255, y el CHK resultante tiene patrones de bits complejos que frecuentemente incluyen los bits D1 y D2 en estado alto, haciendo que el glitch sea detectable.

2.9.5 Versión 4 — Soluciones implementadas

La versión 4.0.0 implementa cuatro correcciones:

1. **[FIX-1] Reasignación del bus de datos:** los bits D1 y D2 del bus se reasignan a los pines A2 y A3 del Arduino, que son GPIO puramente digitales sin ninguna

función de timer en el ATmega328P. Esta es la corrección principal: elimina físicamente la fuente de interferencia. Los pines A2 y A3 son entradas analógicas del ADC (Analog-to-Digital Converter), pero también pueden configurarse como GPIO digital estándar. Al no tener ningún timer asociado, son inmunes a las conmutaciones del PWM.

2. **[FIX-2] Doble lectura de verificación:** tanto `escribirByte()` (en D1) como `leerBus()` (en D2) implementan doble lectura con 50 μ s de espera intermedia. Esta es una defensa adicional que detecta y corrige glitches de corta duración en cualquier pin del bus.
3. **[FIX-3] Hold time aumentado:** de 5 ms a 8 ms. Margen adicional para que D2 capture el dato en cualquier condición de carga del loop.
4. **[FIX-4] Inter-frame aumentado:** de 3 ms a 5 ms. Da más tiempo a D2 para completar `analogWrite()` antes de que el bus cambie al siguiente dato.

Con estas correcciones, la versión v4.0.0 logró **0% de errores de checksum** en todas las pruebas realizadas, tanto en tramas ON/OFF como en tramas COLOR con valores aleatorios.

Versión	Setup/Hold	Error observado	Corrección aplicada
v1	0 ms / 0 ms	Errores frecuentes, aleatorios	Introducir timing básico
v2	5 ms / 3 ms	Error 0x06 sistemático en T4 COLOR	Aumentar tiempos + logging
v3	10 ms / 5 ms	Error 0x06 persiste en T4 COLOR	Análisis forense → crosstalk PWM
v4	15 ms / 8 ms	Sin errores (0%)	Reasignación pines + doble lectura

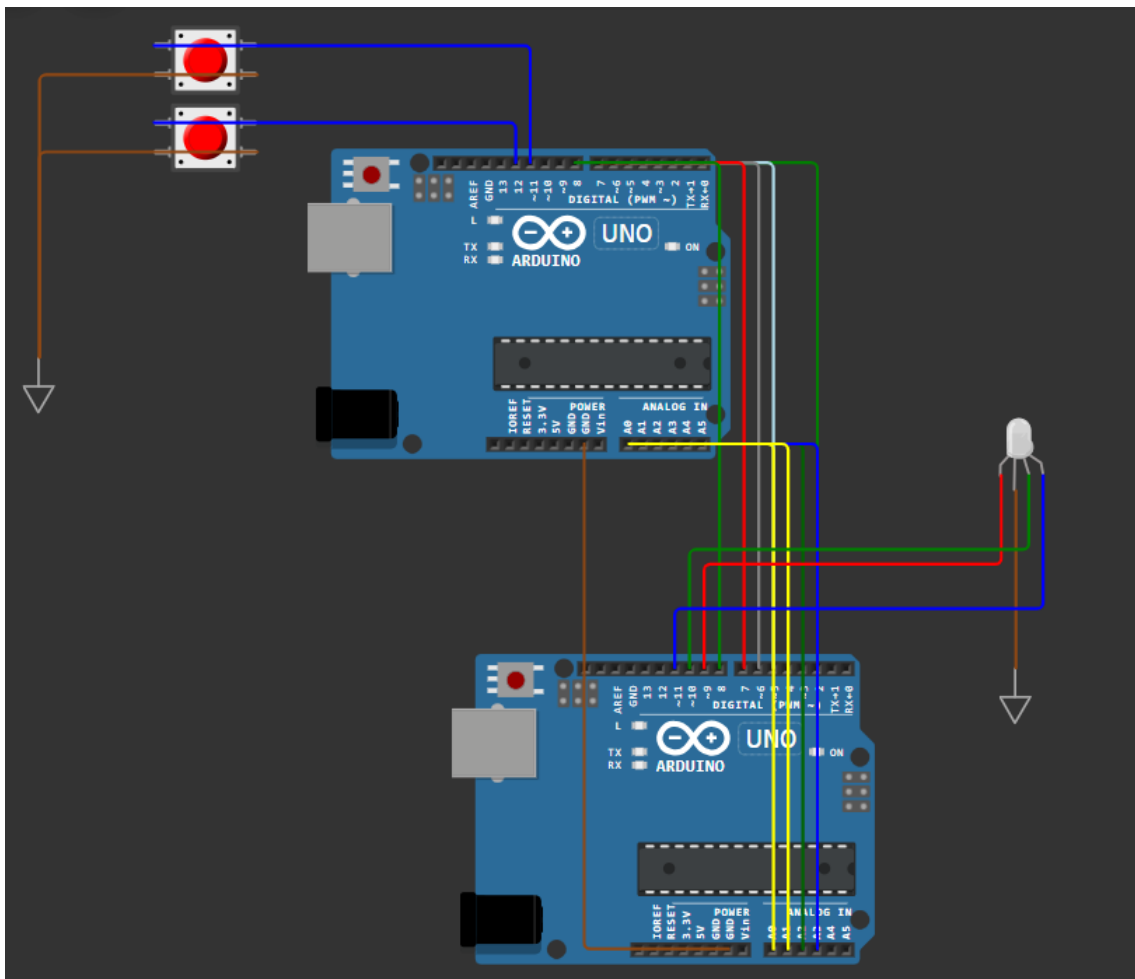
2.10 Conexión eléctrica y esquema de circuito

La conexión física requiere 10 conductores entre los dos dispositivos: 8 cables de datos (D0–D7), 1 cable de strobe (STR) y 1 cable de referencia de masa (GND común). Este último es crítico: sin una referencia de masa común, los niveles lógicos HIGH y LOW no son relativos a la misma referencia y la comunicación es imposible.

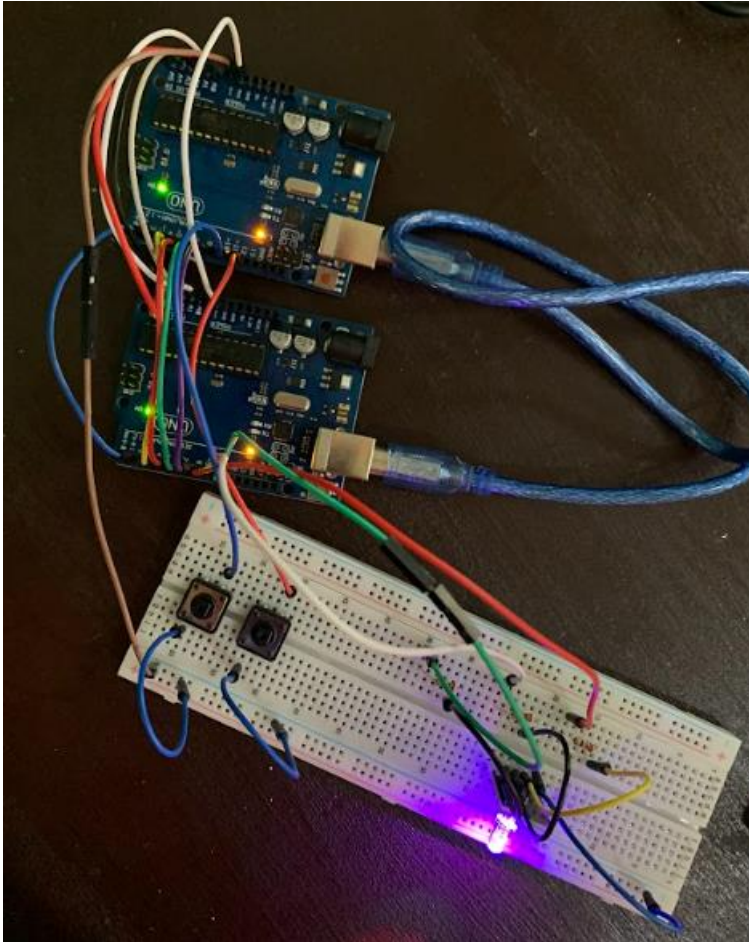
- D1.pin2 → D2.pin2: D0 del bus
- D1.pinA2 → D2.pinA2: D1 del bus (nuevo en v4, antes pin 3)

- D1.pinA3 → D2.pinA3: D2 del bus (nuevo en v4, antes pin 4)
- D1.pin5 → D2.pin5: D3 del bus
- D1.pin6 → D2.pin6: D4 del bus
- D1.pin7 → D2.pin7: D5 del bus
- D1.pin8 → D2.pin8: D6 del bus
- D1.pinA0 → D2.pinA0: D7 del bus
- D1.pinA1 → D2.pinA1: STR — strobe
- D1.GND → D2.GND: referencia común — CRÍTICO

El LED RGB se conecta a los pines PWM de D2 (pines 9, 10 y 11) mediante resistencias limitadoras de corriente de 220 Ω . Para LED tipo cátodo común, el cátodo se conecta a GND. Para LED tipo ánodo común, el ánodo se conecta a 5V y la lógica de control se invierte (255 = apagado, 0 = máxima intensidad).



Esquema eléctrico realizado en WOKWI.



Circuito físico funcional ejercicio 1 (comunicación paralela).

2.11 Resultados

El sistema de comunicación paralela v4.0.0 alcanzó los siguientes resultados en las pruebas de validación:

- **Tasa de éxito de transmisión:** 100% (0 errores de checksum en todas las pruebas)
- **Duración por trama:** 36 ms (consistente con el timing teórico)
- **Duración de transmisión completa:** ~180 ms (5 tramas)
- **Reintentos de escritura:** 0 (bus estable con nueva asignación de pines)
- **Reintentos de lectura:** 0 (doble lectura siempre consistente)
- **Comportamiento ante error:** si el checksum falla, el estado del LED no se modifica y el log identifica los bits en error

En el apartado 8. Se adjuntan los link's a los videos que muestran el funcionamiento del ejercicio 1.

3. Ejercicio 2: Comunicación Serie Bidireccional UART

3.1 Descripción del sistema y requisitos

El ejercicio 2 establece una comunicación serie bidireccional entre dos Arduino UNO. El Dispositivo 1 (D1) actúa como **maestro de comandos** y cuenta con dos pulsadores (BTN_ONOFF y BTN_CAMBIAR), LED1 (indicador de ACK) y LED2 (indicador de conexión activa). El Dispositivo 2 (D2) actúa como **receptor y controlador del LED RGB** y cuenta con un pulsador de conexión (BTN_CONEXION) y el LED RGB.

El sistema implementa un handshake de conexión explícito: D2 debe solicitar conexión a D1 presionando BTN_CONEXION, y D1 debe aceptarla respondiendo con CONN_ACK. Solo tras esta negociación, D1 puede enviar comandos. Cada comando enviado es confirmado por D2 con un ACK (recepción correcta) o un NACK (checksum incorrecto), y D1 enciende LED1 durante 2 segundos al recibir un ACK válido.

3.2 El periférico UART y la biblioteca SoftwareSerial

UART (Universal Asynchronous Receiver-Transmitter) es el protocolo serie asincrónico más ampliamente utilizado en sistemas embebidos. En UART, los datos se transmiten en forma serie (bit a bit) sin una señal de reloj compartida: el sincronismo se logra mediante una velocidad de transmisión previamente acordada (baud rate) y bits de start/stop que enmarcan cada byte.

El ATmega328P del Arduino UNO dispone de un único periférico UART hardware (USART0) en los pines 0 (RX) y 1 (TX). Sin embargo, este puerto está compartido con el convertidor USB-Serie CH340 o ATmega16U2 que proporciona la conectividad USB del IDE. Usar el UART hardware para la comunicación entre dispositivos mientras se usa el monitor serie del IDE es imposible, ya que ambas funciones comparten los mismos pines físicos.

Para resolver esta limitación, el firmware utiliza la biblioteca **SoftwareSerial**, incluida en el Arduino IDE estándar. SoftwareSerial emula un puerto UART por software en cualquier par de pines GPIO, utilizando interrupciones y temporizado por software para generar y detectar los bits de start, datos y stop. Esto libera el UART hardware exclusivamente para el monitor de debug, replicando la metodología del ejercicio 1 (bus paralelo + Serial).

3.2.1 Configuración de SoftwareSerial

La instancia de SoftwareSerial se crea con los pines RX y TX como argumentos:

```

SoftwareSerial serieD2(4, 5); // D1: RX=pin4, TX=pin5
SoftwareSerial serieD1(4, 5); // D2: RX=pin4, TX=pin5

// Conexión cruzada entre dispositivos:
// D1.pin5 (TX) → D2.pin4 (RX)
// D2.pin5 (TX) → D1.pin4 (RX)

```

El baud rate seleccionado es **9600 bps**. Esta velocidad se justifica por tres razones: (1) es suficiente para las tramas cortas del protocolo (máximo 5 bytes = $5 \times 10 \text{ bits} / 9600 = \sim 5.2 \text{ ms}$ por trama completa); (2) es ampliamente compatible con SoftwareSerial sin problemas de sincronismo; y (3) minimiza los errores de temporizado por software inherentes a SoftwareSerial (a velocidades mayores, la emulación por software pierde precisión).

La inicialización del puerto serie se realiza llamando a `serieD2.begin(BAUD_SERIE)` en `setup()`. Los métodos fundamentales son: `serieD2.write(byte)` para transmitir un byte, `serieD2.available()` para verificar si hay bytes disponibles en el buffer de recepción, y `serieD2.read()` para leer el siguiente byte del buffer.

3.3 Protocolo serie: tramas y handshake

El protocolo define 6 tipos de tramas con headers únicos para facilitar su identificación en el canal:

Tipo	Header	Dirección	Bytes totales	Descripción
CONN_REQ	0xAA	D2 → D1	2	Solicitud de conexión. D2 inicia el handshake.
CONN_ACK	0xBB	D1 → D2	2	Aceptación de conexión. D1 habilita la sesión.
CMD_ONOFF	0x01	D1 → D2	3	Encender/apagar LED RGB. Byte 2: STATE (0x01=ENC, 0x00=APG).
CMD_COLOR	0x02	D1 → D2	5	Cambiar color RGB. Bytes 2–4: R, G, B (0–255 cada uno).
ACK	0xFF	D2 → D1	2	Confirmación positiva. CHK correcto, comando ejecutado.
NACK	0xFE	D2 → D1	2	Rechazo. CHK incorrecto, comando descartado. LED sin cambio.

El último byte de cada trama es siempre el checksum, calculado sobre todos los bytes previos (incluyendo el header). La estructura general es: `[HEADER] [PAYLOAD...] [CHK]`.

La elección de los valores de header fue deliberada: 0xAA y 0xBB son patrones de bits alternados fácilmente identificables en un osciloscopio; 0xFF y 0xFE son el valor máximo y el penúltimo, diferenciables en una sola comparación; 0x01 y 0x02 son los órdenes de comando más simples posibles, minimizando el riesgo de confusión con datos.

3.3.1 Proceso de handshake de conexión

La secuencia de conexión sigue los siguientes pasos:

5. El usuario presiona `BTN_CONEXION` en D2.
6. D2 envía `CONN_REQ`: `[ 0xAA | CHK ] = [ 0xAA | 0xAA ]` por `SoftwareSerial`.
7. D1, en estado `IDLE`, detecta el byte 0xAA en el canal, verifica el checksum y, si es correcto, envía `CONN_ACK`: `[ 0xBB | CHK ] = [ 0xBB | 0xBB ]`.
8. D1 enciende `LED2` permanentemente y transiciona a estado `CONECTADO`.
9. D2 recibe `CONN_ACK`, verifica el checksum y transiciona a estado `CONECTADO`.
10. El sistema está listo para recibir comandos.

Si D1 recibe un `CONN_REQ` con checksum incorrecto, rechaza la conexión (no envía `CONN_ACK`) y permanece en `IDLE`. Si D2 no recibe `CONN_ACK` dentro del timeout (1500 ms), muestra el diagnóstico correspondiente y aguarda una nueva pulsación de `BTN_CONEXION`.

3.4 Máquina de estados de D1 y D2

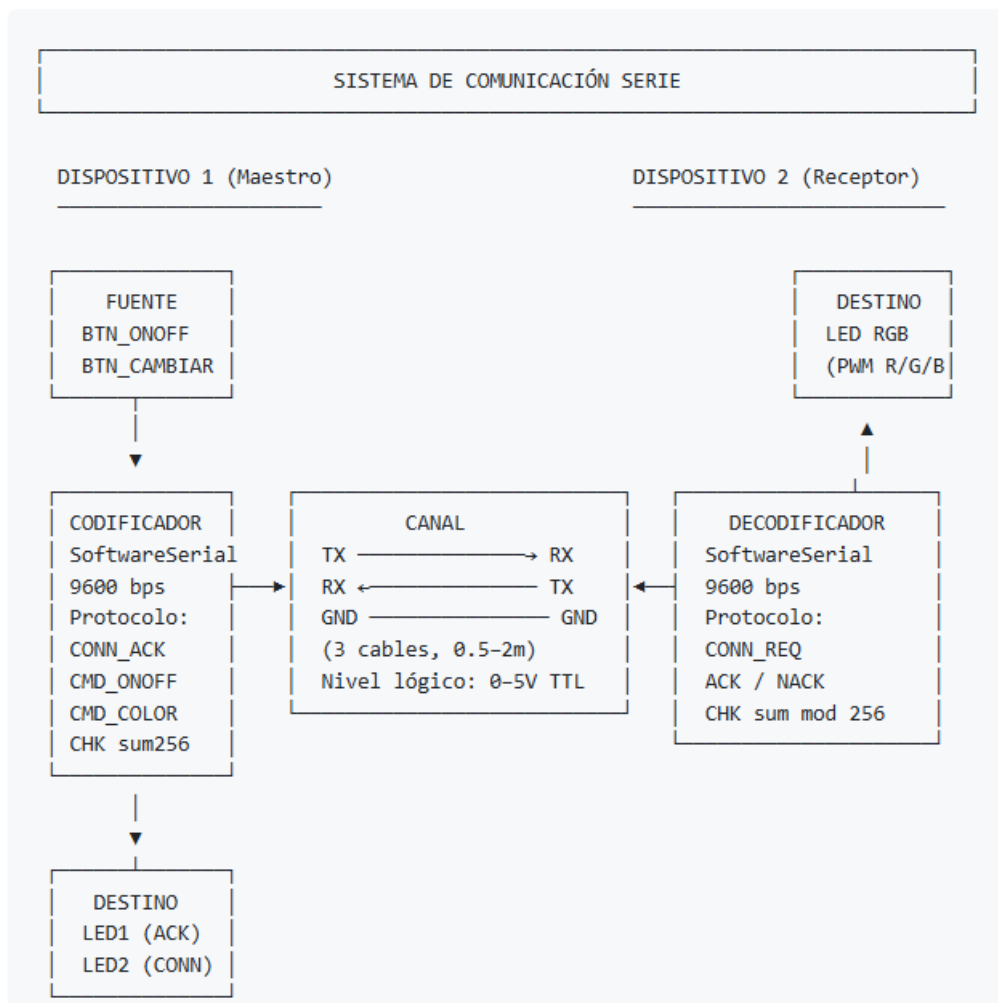
Ambos dispositivos implementan máquinas de estados explícitas mediante un `switch` en el loop principal sobre la variable `estadoSistema`.

Máquina de estados de D1:

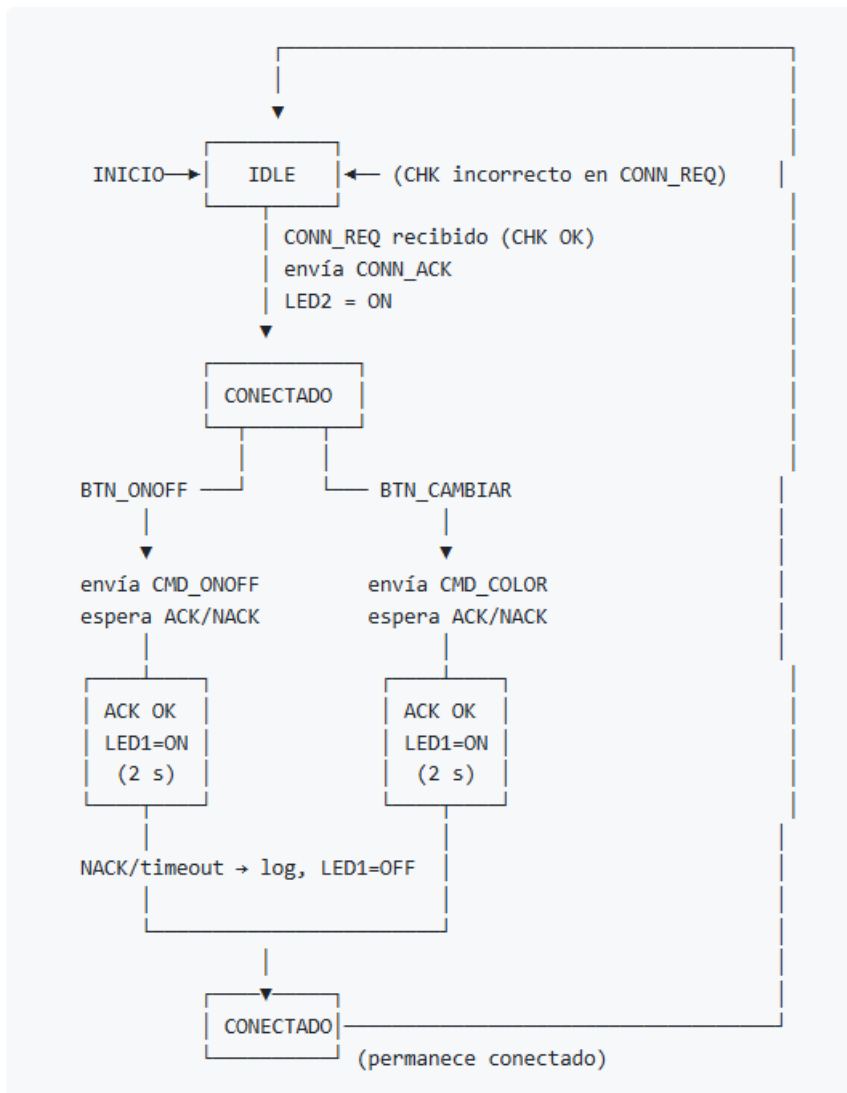
- **IDLE (0):** D1 escucha el canal mediante `serieD1.available()`. Al recibir el primer byte, si es 0xAA (`CONN_REQ`) lee el `CHK` y lo verifica. Si `CHK` OK, envía `CONN_ACK`, enciende `LED2` y transiciona a `CONECTADO`.
- **CONECTADO (1):** D1 monitorea los botones `BTN_ONOFF` y `BTN_CAMBIAR`. Cada pulsación construye y envía la trama correspondiente, luego llama a `procesarRespuesta()` que espera el `ACK/NACK` de D2.

Máquina de estados de D2:

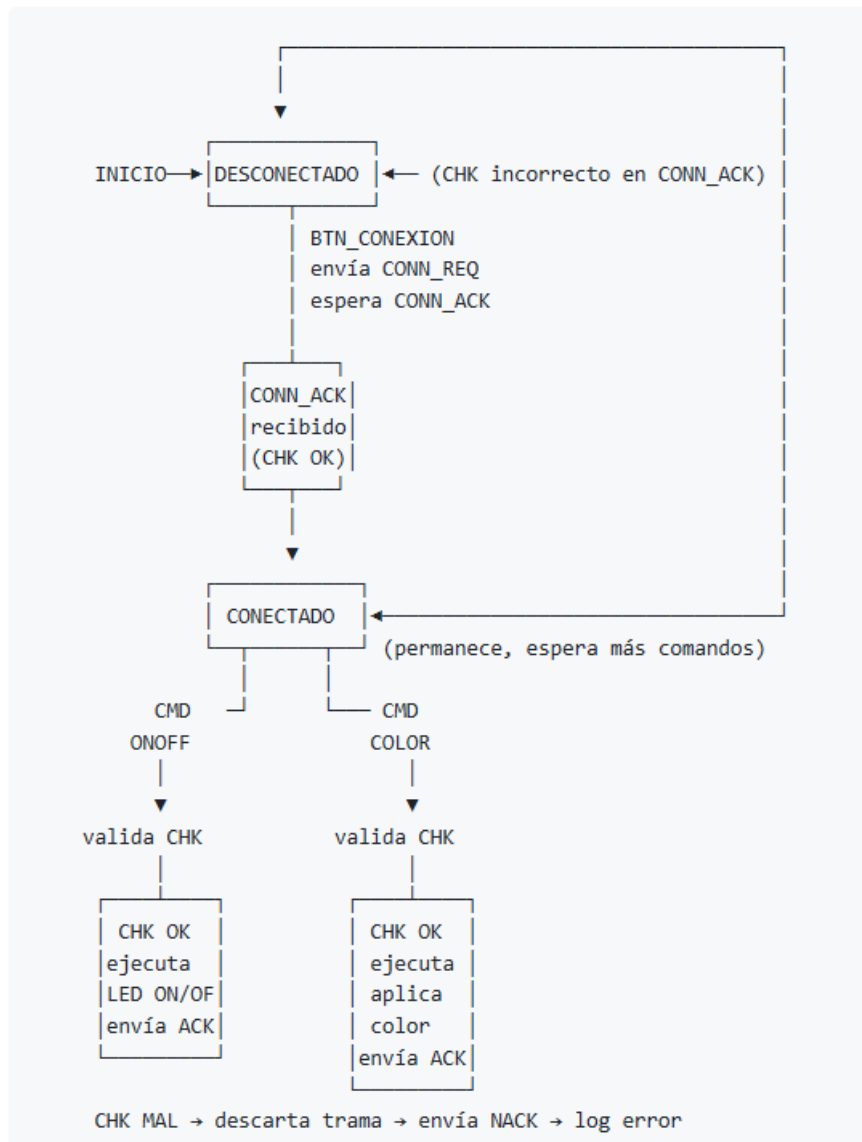
- **DESCONECTADO (0):** D2 espera pulsación de BTN_CONEXION. Al detectarla, llama a `solicitarConexion()` que envía `CONN_REQ` y espera `CONN_ACK`.
- **CONECTADO (1):** D2 monitorea el canal con `serieD1.available()`. Al detectar bytes disponibles, llama a `procesarComando()` que lee el header, lee el payload, verifica CHK y ejecuta el comando.

2. Esquema en bloques de la comunicación

Dispositivo 1 — Máquina de estados



Dispositivo 2 — Máquina de estados



3.5 Checksum por suma módulo 256: fundamento teórico

El ejercicio 2 utiliza un mecanismo de checksum diferente al del ejercicio 1. En lugar de XOR, se implementa la **suma de todos los bytes del payload, módulo 256**: $CHK = (B_0 + B_1 + \dots + B_n) \& 0xFF$.

Ventajas frente al XOR del ejercicio 1:

- Detecta todos los errores de un solo byte alterado (el XOR también, pero la suma proporciona mayor sensibilidad a errores de magnitud: si un byte cambia de 0x10 a 0x11, la suma detecta el error por diferencia de 1 unidad).
- Detecta la mayoría de errores de doble byte, excepto cuando la suma de los cambios es múltiplo de 256 (caso extremadamente raro en la práctica).

- Es el mecanismo base de protocolos industriales como Modbus RTU (donde se usa LRC = complemento de la suma).
- Permite mostrar el cálculo paso a paso en el log ($0xAA + 0x01 + \dots = 0xFF \rightarrow \text{mod}256 = 0x00$), facilitando la comprensión didáctica.

Limitaciones comunes con el XOR:

- No detecta reordenamiento de bytes.
- Para mayor robustez se requeriría CRC-16 o superior (Stallings, 2022). CRC-16 detecta todos los errores de 1 bit, todos los errores de 2 bits en tramas de hasta 32.767 bits, y todos los errores de ráfaga de hasta 16 bits.

La suma módulo 256 se justifica en el ejercicio 2 porque el canal serie (SoftwareSerial) es más vulnerable al ruido que el bus paralelo de corta distancia del ejercicio 1: los errores típicos en UART son errores de byte completo (bit de start/stop mal interpretado), que tanto el XOR como la suma detectan con certeza. Además, la mayor variedad de tramas (CONN_REQ, CONN_ACK, CMD_ONOFF, CMD_COLOR, ACK, NACK) hace que la suma sea más representativa de la diversidad de valores.

3.5.1 Comportamiento ante una trama corrupta

Cuando D2 recibe una trama con checksum incorrecto:

11. Calcula el CHK local sobre los bytes de payload recibidos.
12. Compara con el CHK recibido (último byte de la trama).
13. Si difieren: descarta la trama sin ejecutar ningún comando. El estado del LED RGB permanece sin cambios.
14. Envía NACK ([0xFE | CHK(0xFE)]) a D1.
15. Loguea el error con el detalle de CHK recibido vs. calculado, la diferencia absoluta y las posibles causas (ruido eléctrico, GND no común, baud rate distinto, SoftwareSerial con interferencias).

Cuando D1 recibe el NACK: no reenvía el comando (ARQ no implementado en esta versión, fuera del alcance de la consigna) y loguea el evento. LED1 **no enciende**, ya que solo lo hace ante un ACK válido.

3.6 Implementación del firmware — Dispositivo 1 (Maestro)

El firmware del Dispositivo 1 (archivo `dispositivo1_serie.ino`) implementa las siguientes funciones principales:

3.6.1 Envío de tramas con checksum

La función `enviarTrama(datos, longitud)` transmite el array de bytes de payload y agrega automáticamente el CHK calculado como último byte. El checksum se calcula internamente mediante `calcularChecksum()`, por lo que el array `datos` no debe incluirlo. Cada byte se envía mediante `serieD2.write()`.

```
void enviarTrama(uint8_t* datos, uint8_t longitud) {
    uint8_t chk = calcularChecksum(datos, longitud);
    for (uint8_t i = 0; i < longitud; i++) {
        serieD2.write(datos[i]);
    }
    serieD2.write(chk); // CHK adjunto al final
}
```

3.6.2 Recepción con timeout

La función `recibirTrama(buffer, esperados, timeoutMs)` espera exactamente `esperados` bytes del canal dentro del timeout especificado. Utiliza polling de `serieD2.available()` con verificación de tiempo mediante `millis()`. Si el canal está silencioso más de `timeoutMs` ms, retorna `false` y loguea el error con diagnóstico. El timeout de ACK está configurado en 3000 ms (`TIMEOUT_ACK_MS`).

3.6.3 Procesamiento de respuesta ACK/NACK

La función `procesarRespuesta()` se llama después de cada comando enviado. Espera 2 bytes (HEADER + CHK), verifica la integridad del ACK/NACK recibido, y toma la acción correspondiente: si es ACK válido, llama a `encenderLED1()` que mantiene el LED1 encendido durante `LED1_DURACION_MS` (2000 ms); si es NACK, loguea el error; si es timeout, incrementa el contador de timeouts.

3.7 Implementación del firmware — Dispositivo 2 (Receptor/Esclavo)

El firmware del Dispositivo 2 (archivo `dispositivo2_serie.ino`) implementa las siguientes funciones principales:

3.7.1 Solicitud de conexión

La función `solicitarConexion()` gestiona el handshake iniciado por D2. Envía `CONN_REQ`, espera `CONN_ACK` dentro de `TIMEOUT_CONN_ACK_MS` (1500 ms), y verifica el header (0xBB) y el CHK. Si la conexión es exitosa, transiciona a `ESTADO_CONECTADO`. La variable estática `intentos` lleva la cuenta de reintentos para el log.

3.7.2 Procesamiento de comandos

La función `procesarComando()` lee el byte de header del canal y lo usa como discriminador:

- `0x01 (CMD_ONOFF)`: lee 2 bytes más (STATE + CHK), verifica el CHK, y si es correcto llama a `ejecutarOnOff(estado)` que enciende o apaga el LED RGB. Luego envía ACK. Si CHK falla, envía NACK.
- `0x02 (CMD_COLOR)`: lee 4 bytes más (R + G + B + CHK), verifica el CHK, y si es correcto llama a `ejecutarColor(r,g,b)` que actualiza el color del LED. Luego envía ACK. Si CHK falla, envía NACK.
- Header desconocido: byte ignorado, log de warning con diagnóstico.

3.7.3 Gestión del estado del LED RGB

D2 mantiene las variables `ledEncendido`, `colorActualR`, `colorActualG` y `colorActualB` para preservar el estado del LED. Cuando se recibe CMD_COLOR con el LED apagado, el color se guarda, pero no se aplica al PWM (se aplicará al recibir CMD_ONOFF con STATE=1). Cuando se recibe CMD_ONOFF con STATE=0 (apagar), `apagarLED()` pone los tres canales PWM a 0 pero no borra el color guardado, de modo que al volver a encender se restaura el último color.

3.8 Sistema de logging y diagnóstico

El sistema de logging del ejercicio 2 sigue los mismos principios que el del ejercicio 1, pero adaptado al protocolo serie:

- **Log de transmisión TX:** para cada trama enviada, imprime el tipo, cada byte con su valor hex/decimal y descripción de campo, y el cálculo detallado del checksum (suma paso a paso → módulo 256 → resultado).
- **Log de recepción RX:** para cada trama recibida, imprime el tipo identificado, cada byte con descripción de campo, y la verificación del CHK con resultado OK o ERROR.
- **Log de handshake:** documenta cada etapa del proceso CONN_REQ / CONN_ACK con todos los bytes involucrados.
- **Log de errores de checksum:** diferencia absoluta entre CHK recibido y calculado, posibles causas (ruido, GND, baud rate, SoftwareSerial).
- **Log de timeout:** número de bytes recibidos vs. esperados, y diagnóstico del cable RX afectado.

- **Contadores de sesión:** TX, RX, ACKs, NACKs, errores CHK, timeouts, y tasa de éxito porcentual.

Log completo de inicio (D1) CONN_REQ, ON/OFF y CAMBIAR

```
=====
DISPOSITIVO 1 | Maestro | UART Serie
Comunicación de Datos – Ejercicio 2 | v1.0.0
=====

Baud SoftSerial   : 9600
PINOUT SoftSerial: RX=4, TX=5
BTN ON/OFF       : pin 11
BTN CAMBIAR      : pin 12
LED1 (ACK)       : pin 8
LED2 (CONEXION)  : pin 9
Checksum         : Suma de bytes MOD 256
NACK             : sin reenvio automatico (ARQ no implementado)
=====

[INIT] Estado inicial: IDLE
[INIT] Esperando solicitud de conexion de D2...

-----

[RX] CONN_REQ recibido: AA AA
[RX] Trama CONN_REQ:
    Byte 0: 0xAA (dec=170)
    Byte 1: 0xAA (dec=170)
    Verificacion CHK: recibido=0xAA | calculado=0xAA | OK ✓
-----

[TX #0] Enviando CONN_ACK
    Byte 0: 0xBB (dec=187) ← HEADER CONN_ACK
    CHK (Suma mod256): 0xBB = 0xBB → mod256 = 0xBB
    Byte CHK: 0xBB (adjuntado al final de la trama)
    --> Trama enviada por SoftwareSerial.
[LED2] Encendido – conexion activa
[ESTADO] Sistema CONECTADO. Esperando botones...
-----
```



```
[BTN] ON/OFF presionado
      Estado nuevo del LED RGB: ENCENDIDO
-----
[TX #1] Enviando CMD_ONOFF
      Byte 0: 0x01 (dec=1) ← HEADER CMD_ONOFF
      Byte 1: 0x01 (dec=1) ← STATE (1=ENC, 0=APG) → ENCENDER
      CHK (Suma mod256): 0x01 + 0x01 = 0x02 → mod256 = 0x02
      Byte CHK: 0x02 (adjuntado al final de la trama)
      --> Trama enviada por SoftwareSerial.
[ESPERA] Esperando ACK/NACK de D2... (timeout: 3000 ms)
[RX] Trama ACK:
      Byte 0: 0xFF (dec=255)
      Byte 1: 0xFF (dec=255)
      Verificacion CHK: recibido=0xFF | calculado=0xFF | OK ✓
[RX] ACK valido recibido para CMD_ONOFF
[LED1] Encendido por 2000 ms (ACK confirmado)
[LED1] Apagado
[BTN] CAMBIAR presionado
      Color aleatorio generado: R=107 G=3 B=77
-----
[TX #2] Enviando CMD_COLOR
      Byte 0: 0x02 (dec=2) ← HEADER CMD_COLOR
      Byte 1: 0x6B (dec=107) ← R (Rojo)
      Byte 2: 0x03 (dec=3) ← G (Verde)
      Byte 3: 0x4D (dec=77) ← B (Azul)
      CHK (Suma mod256): 0x02 + 0x6B + 0x03 + 0x4D = 0xBD → mod256 = 0xBD
      Byte CHK: 0xBD (adjuntado al final de la trama)
      --> Trama enviada por SoftwareSerial.
[ESPERA] Esperando ACK/NACK de D2... (timeout: 3000 ms)
[RX] Trama ACK:
      Byte 0: 0xFF (dec=255)
      Byte 1: 0xFF (dec=255)
      Verificacion CHK: recibido=0xFF | calculado=0xFF | OK ✓
[RX] ACK valido recibido para CMD_COLOR
[LED1] Encendido por 2000 ms (ACK confirmado)
[LED1] Apagado
```

Log completo de una recepción (D2)

```
=====
DISPOSITIVO 2 | Receptor | UART Serie
Comunicacion de Datos – Ejercicio 2 | v1.0.0
=====

Baud SoftSerial : 9600
PINOUT SoftSerial: RX=4, TX=5
LED RGB         : R=9(PWM) G=10(PWM) B=6(PWM)
BTN CONEXION    : pin 11
Checksum        : Suma de bytes MOD 256
NACK            : trama corrupta descartada, sin reenvio
=====

[INIT] Estado inicial: DESCONECTADO
[INIT] Presione BTN_CONEXION para iniciar handshake con D1.

[BTN] CONEXION presionado. Iniciando handshake...
-----
[TX] CONN_REQ – intento #1
[TX] Enviando CONN_REQ
    Byte 0: 0xAA (dec=170) ← HEADER CONN_REQ (solicitud conexion)
    CHK: suma(0xAA)=0xAA → mod256=0xAA
    --> Trama enviada.
[ESPERA] Esperando CONN_ACK... (timeout: 1500 ms)
    Tipo identificado: CONN_ACK
    Byte 0: 0xBB (dec=187) ← HEADER
    Byte 1: 0xBB (dec=187) ← CHK recibido
    Verificacion CHK (suma mod256): 0xBB = 0xBB → mod256 = 0xBB | Recibido: 0xBB → OK ✓
[ESTADO] Conexion establecida con D1. Sistema CONECTADO.
    Esperando comandos CMD_ONOFF / CMD_COLOR...
-----
```

```
-----
[RX #1] Header recibido: 0x01
  Tipo identificado: CMD_ONOFF
  Byte 0: 0x01 (dec=1) ← HEADER
  Byte 1: 0x01 (dec=1) ← STATE: ENCENDER (0x01)
  Byte 2: 0x02 (dec=2) ← CHK recibido
  Verificacion CHK (suma mod256): 0x01+0x01 = 0x02 → mod256 = 0x02 | Recibido: 0x02 → OK ✓
[CMD] ON/OFF → LED RGB: ENCENDIDO
  Restaurando color anterior: R=0 G=0 B=0
  [LED-RGB] PWM aplicado: R=0 G=0 B=0
[TX] Enviando ACK
  Byte 0: 0xFF (dec=255) ← HEADER ACK (confirmacion OK)
  CHK: suma(0xFF)=0xFF → mod256=0xFF
  --> Trama enviada.
-----

[RX #2] Header recibido: 0x02
  Tipo identificado: CMD_COLOR
  Byte 0: 0x02 (dec=2) ← HEADER
  Byte 1: 0x6B (dec=107) ← R (Rojo) = 107
  Byte 2: 0x03 (dec=3) ← G (Verde) = 3
  Byte 3: 0x4D (dec=77) ← B (Azul) = 77
  Byte 4: 0xBD (dec=189) ← CHK recibido
  Verificacion CHK (suma mod256): 0x02+0x6B+0x03+0x4D = 0xBD → mod256 = 0xBD | Recibido: 0xBD → OK ✓
[CMD] COLOR → R=107 G=3 B=77
  [LED-RGB] PWM aplicado: R=107 G=3 B=77
[TX] Enviando ACK
  Byte 0: 0xFF (dec=255) ← HEADER ACK (confirmacion OK)
  CHK: suma(0xFF)=0xFF → mod256=0xFF
  --> Trama enviada.
```

3.9 Comparación con protocolo alternativo: SPI

El protocolo alternativo más relevante para este escenario es **SPI (Serial Peripheral Interface)**, un protocolo serie síncrono que utiliza cuatro cables: MOSI (datos maestro→esclavo), MISO (datos esclavo→maestro), SCK (reloj) y CS/SS (selección de esclavo).

Aspecto	UART (SoftwareSerial 9600 bps)	SPI (hardware, hasta 8 Mbps)
Tipo	Asíncrono	Síncrono (requiere reloj compartido SCK)
Nº cables	2 (TX, RX) + GND = 3	4 (MOSI, MISO, SCK, CS) + GND = 5
Velocidad	9600 bps (SoftwareSerial)	Hasta 8 Mbps (hardware)
Bidireccional	Sí, full-duplex nativo	Sí, full-duplex nativo
Detección errores	CHK por suma mod256 (implementado)	Ninguna nativa (requiere implementación)
Sincronismo	Por bits de start/stop (autosuficiente)	Requiere reloj SCK siempre activo
Facilidad impl.	Alta: disponible en Arduino IDE estándar	Media: requiere configuración de periférico SPI

Aspecto	UART (SoftwareSerial 9600 bps)	SPI (hardware, hasta 8 Mbps)
Uso adecuado	Distancias cortas, bajo caudal de datos	Alta velocidad, distancias muy cortas (<10 cm)

La elección de UART (mediante SoftwareSerial) sobre SPI para este escenario se justifica por las siguientes razones:

16. **Eficiencia del cableado:** UART requiere solo 2 cables de datos (TX y RX) frente a los 4 de SPI.
17. **Velocidad suficiente:** 9600 bps es más que suficiente para las tramas cortas del protocolo (máximo 5 bytes = ~5.2 ms por trama). La velocidad de SPI no aporta ventaja para este caso de uso.
18. **Robustez frente a errores:** UART incluye por protocolo bits de start y stop que permiten al receptor sincronizarse con el transmisor incluso después de un error. SPI, al ser síncrono, requiere que el reloj SCK nunca falle; si se pierde un ciclo de SCK, toda la comunicación se desincroniza.
19. **Soporte nativo en Arduino IDE:** SoftwareSerial está incluida en el IDE estándar y no requiere configuración de registros de hardware. Esto facilita la implementación y el debugging.

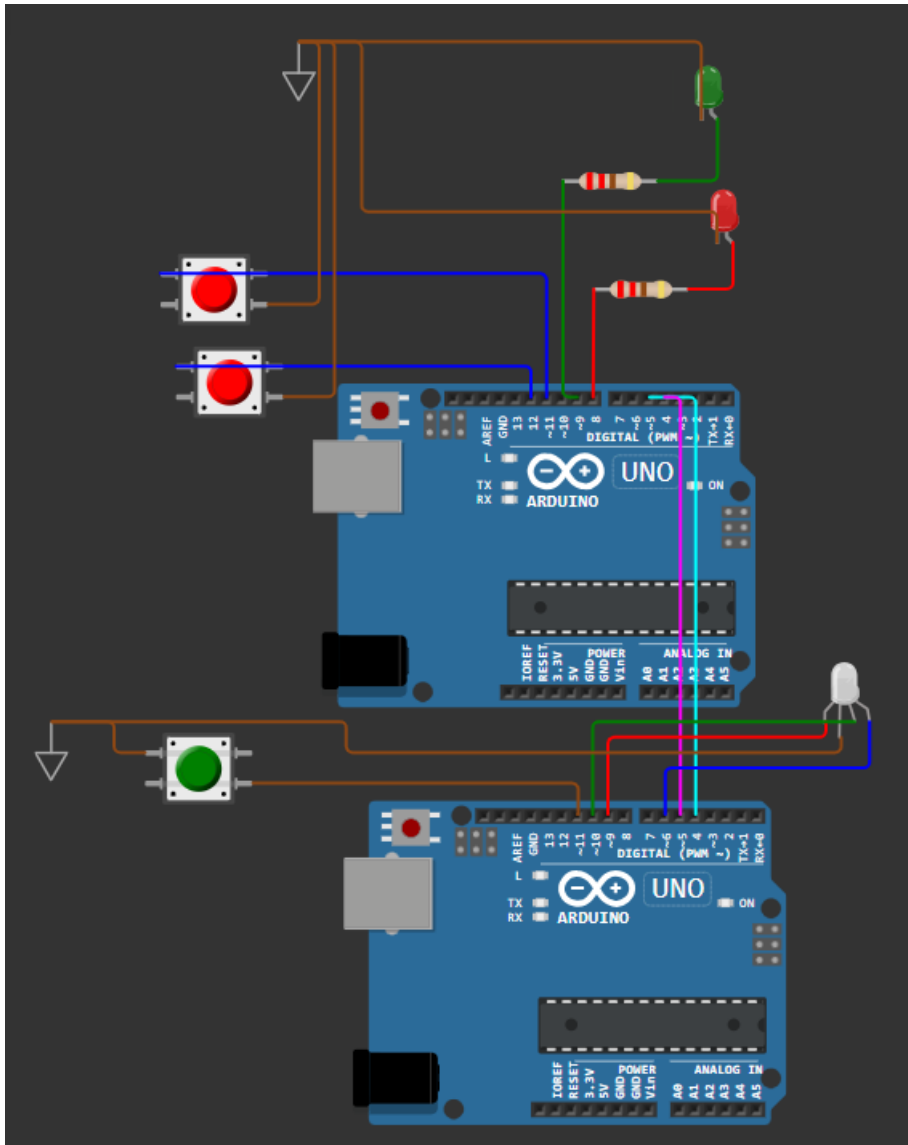
3.10 Conexión eléctrica y esquema de circuito

La conexión física del ejercicio 2 requiere 3 conductores entre los dispositivos:

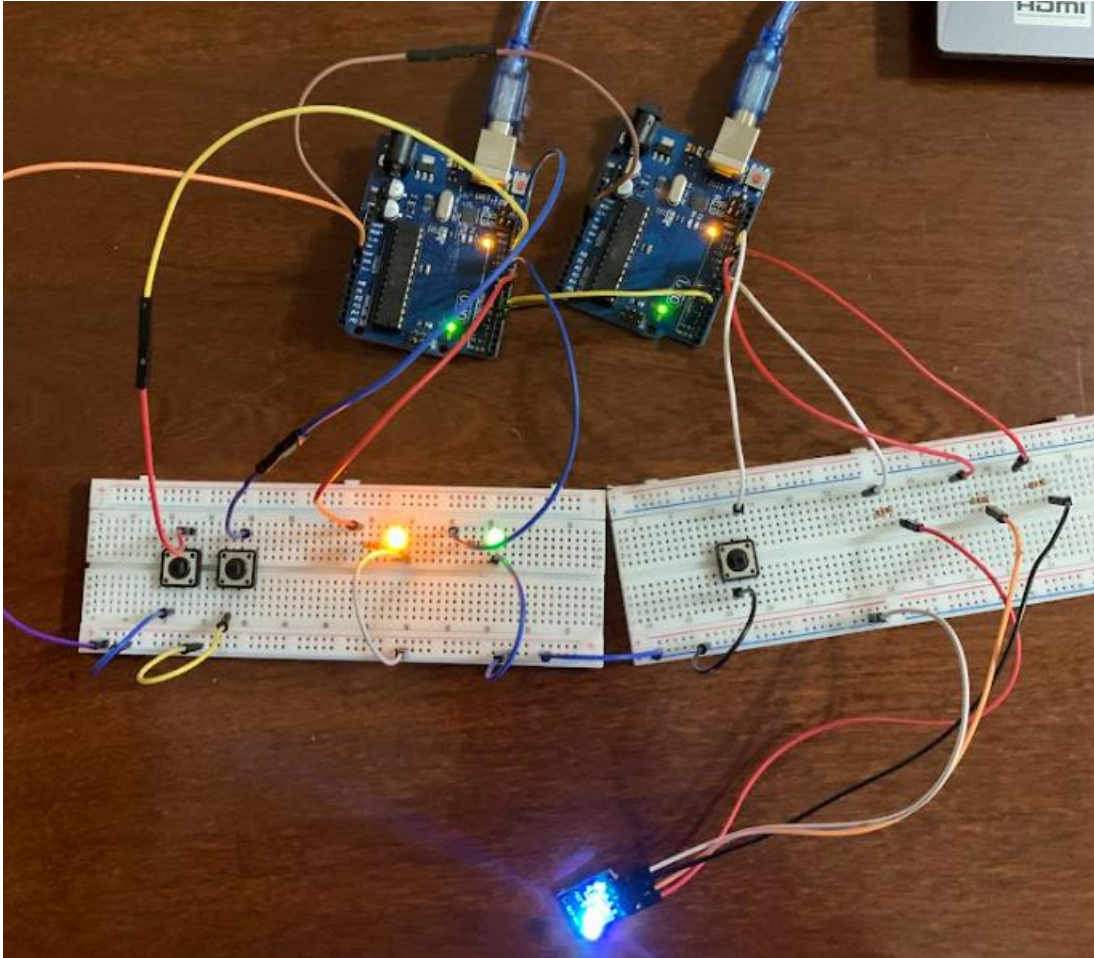
- D1.pin5 (TX) → D2.pin4 (RX): datos de D1 hacia D2
- D2.pin5 (TX) → D1.pin4 (RX): datos de D2 hacia D1 (comunicación bidireccional)
- GND común: referencia de tensión compartida (OBLIGATORIO)

El LED RGB en D2 se conecta a pines PWM: R=9, G=10, B=6 (pin 6 en lugar de 11 para liberar el pin 11 al botón BTN_CONEXION, ya que el pin 6 también tiene PWM por Timer0 en el UNO). Las resistencias limitadoras de 220 Ω protegen los LEDs de exceso de corriente.

Los LEDs de D1 (LED1 indicador de ACK, LED2 indicador de conexión) se conectan a los pines 8 y 9 respectivamente, cada uno con una resistencia de 220 Ω hacia GND.



Esquema eléctrico ejercicio 2 (comunicación serie bidireccional UART).



Circuito físico funcional – Ejercicio 2, Comunicación Serie UART.

3.11 Resultados

- **Handshake:** establecimiento exitoso de conexión en todos los intentos sin errores de checksum
- **CMD_ONOFF:** encendido y apagado del LED RGB confirmado con ACK en todos los casos
- **CMD_COLOR:** cambio de color exitoso con ACK. El color se preserva al apagar y restaura al volver a encender
- **LED1:** encendido durante exactamente 2 segundos al recibir cada ACK
- **LED2:** permanece encendido durante toda la sesión conectada
- **Detección de NACK:** verificado inyectando manualmente un bit de error en el cable TX; D2 detectó el CHK incorrecto, envió NACK y D1 loggeó el evento correctamente

En sección 8. Se adjunta link a video de demostración de funcionamiento del ejercicio 2.

4. Comparación entre comunicación paralela y serie

La siguiente tabla compara los dos sistemas implementados según los parámetros definidos en la teoría de comunicaciones de datos (Stallings, 2022; Tanenbaum & Wetherall, 2011):

Parámetro	Bus Paralelo (Ej. 1)	UART Serie (Ej. 2)
Tipo de transmisión	Paralela, 8 bits simultáneos	Serie, 1 bit por vez
Dirección	Unidireccional (D1 → D2)	Bidireccional full-duplex
Nº conductores de datos	9 (8 datos + strobe)	2 (TX y RX)
Velocidad de datos	~222 bits/s (8 bits/36 ms)	9600 bits/s
Sincronismo	Sincrónico (señal STR)	Asincrónico (bits start/stop)
Detección de errores	XOR ($T0 \oplus T1 \oplus T2 \oplus T3$)	Suma mod256
Protocolo de aplicación	Bus propio + strobe	UART + protocolo de tramas propio
Handshake	No implementado	Sí: CONN_REQ / CONN_ACK
Confirmación de entrega	No (unidireccional)	Sí: ACK / NACK
Principal ventaja	Velocidad real por ciclo (8 bits simultáneos)	Menor cantidad de cables, bidireccional
Principal desventaja	Mayor nº de cables, sensible a crosstalk	Menor velocidad bruta por conductor

Desde la perspectiva de la teoría de comunicaciones, el bus paralelo ofrece mayor ancho de banda teórico (8 bits por ciclo de strobe) pero exige sincronismo preciso y es más susceptible al crosstalk a medida que aumenta la frecuencia de operación. El enlace serie, aunque transmite un bit a la vez, es más robusto, requiere menos conductores y es la base de la mayoría de los protocolos de comunicación modernos (USB, SATA, PCIe, etc.). Como concluyen Tanenbaum y Wetherall (2011), la tendencia en la industria ha sido hacia la serialización de buses anteriormente paralelos (ej: IDE → SATA, PCI → PCIe) precisamente por estas razones.

5. Dificultades encontradas y soluciones aplicadas

5.1 Interferencia PWM en el bus paralelo (Ejercicio 1)

La principal dificultad del ejercicio 1 fue el error sistemático de checksum en las tramas COLOR, cuya causa fue el crosstalk capacitivo/inductivo entre las señales PWM del LED RGB y los cables del bus de datos (descrito en detalle en la sección 2.9). La resolución requirió un proceso de diagnóstico de tres iteraciones antes de identificar la causa raíz: la asignación incorrecta de D1 al pin 3 (OC2B del Timer2).

La lección aprendida es que **la asignación de pines en sistemas mixtos (digital + PWM) debe considerar explícitamente el mapa de timers del microcontrolador**. En el ATmega328P, los pines 3, 5, 6, 9, 10 y 11 tienen funciones de timer que los hacen susceptibles a interferencias cuando se usan simultáneamente como GPIO estático y como salida PWM en el mismo circuito.

5.2 Jitter del Serial en la zona crítica de transmisión

Un problema encontrado en versiones tempranas fue que llamadas a `Serial.print()` dentro de la función de strobe introducían jitter de varios milisegundos en los GPIO, lo que causaba variaciones en el timing del strobe detectables por D2. La solución fue la separación estricta entre la zona crítica de transmisión (sin Serial) y la zona de logging (post-transmisión), implementada en `transmitirTrama()`.

5.3 Ausencia de GND común

En las pruebas iniciales, ambos ejercicios exhibían errores aleatorios no reproducibles. La causa fue la ausencia de un cable de referencia de masa (GND) entre los dos dispositivos. Sin GND común, los niveles lógicos HIGH y LOW de D1 no son referenciados a la misma tensión que los de D2, y cualquier diferencia de potencial entre las masas de los dos Arduinos (que operan de fuentes USB independientes) se suma como ruido a todas las señales. La solución fue agregar un cable dedicado entre los pines GND de ambos dispositivos, que se documenta como requisito crítico en los comentarios del firmware.

5.4 Limitaciones de SoftwareSerial

SoftwareSerial presenta una limitación importante: no puede recibir y transmitir simultáneamente (half-duplex en la práctica, aunque el protocolo es full-duplex). Además, la recepción por SoftwareSerial deshabilita temporalmente las interrupciones, lo que puede causar problemas si otras interrupciones (como las del UART hardware) generan eventos durante la recepción. En este proyecto, dado que D1 siempre espera

la respuesta de D2 antes de enviar el próximo comando, la colisión de transmisiones no es posible y la limitación no afecta el funcionamiento.

6. Conclusiones

El presente trabajo demostró experimentalmente las diferencias fundamentales entre la comunicación paralela y la comunicación serie, validando los conceptos teóricos de Stallings (2022) y Tanenbaum & Wetherall (2011) en un entorno físico real.

El ejercicio 1 evidenció que el principal desafío de la comunicación paralela no es la lógica del protocolo, sino la integridad eléctrica del bus: el crosstalk entre señales PWM y cables de datos fue una causa de error no obvia que requirió cuatro iteraciones de firmware y un proceso de diagnóstico sistemático para resolver. La solución — reasignar pines de bus a GPIO sin función de timer — es típica de prácticas de diseño de hardware embebido profesional.

El ejercicio 2 demostró que la comunicación serie, aunque más lenta en velocidad bruta, ofrece ventajas significativas en robustez, bidireccionalidad y reducción de cableado. El protocolo de handshake y el mecanismo de ACK/NACK implementan conceptos fundamentales de las capas 2 (enlace de datos) y 4 (transporte) del modelo OSI, en particular el control de flujo y la confirmación de entrega.

Ambos sistemas de logging demostraron ser herramientas esenciales para el desarrollo y diagnóstico, confirmando la importancia de instrumentar adecuadamente los sistemas embebidos desde las primeras etapas del desarrollo.

7. Referencias bibliográficas

Stallings, W. (2022). *Comunicaciones y redes de computadores* (10ma ed.). Pearson Educación.

Tanenbaum, A. S. & Wetherall, D. J. (2011). *Redes de computadoras* (5ta ed.). Pearson Educación.

Atmel/Microchip Technology. (2015). *ATmega328P datasheet*. Recuperado de https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf

Arduino. (2024). *SoftwareSerial library reference*. Recuperado de <https://www.arduino.cc/en/Reference/softwareSerial>

Arduino. (2024). *analogWrite() function reference*. Recuperado de <https://www.arduino.cc/reference/en/language/functions/analog-io/analogwrite/>

8. Link's a videos de funcionamiento de los ejercicios

Ejercicio 1, versión 3.

https://drive.google.com/file/d/1a0Al9rsk0ddXVXtL_GREeFIWxmt4Z33A/view?usp=sharing

- En el video se observa lo crucial que fue el uso de loggins en monitor serial por parte de ambos dispositivos. Para poder detectar que información envía el dispositivo 1, y que información recibe el dispositivo 2.

Ejercicio 1, versión 4.

<https://drive.google.com/file/d/1XJJYMwpYtHV0FHITY2ePO59ls5oUDiXq/view?usp=sharing>

- Se observa como en la última versión corregida del ejercicio, la información enviada por el dispositivo 1, llega de manera correcta hacia el dispositivo 2. Respondiendo de manera acertada ante las peticiones de ON/OFF o CAMBIAR_COLOR.

Ejercicio 2.

https://drive.google.com/file/d/1GWyUIZSmFSOiJcqXj5BdNrzDK_zWcYSk/view?usp=sharing

- Funcionamiento del ejercicio 2.